

60+ Pro Flutter Tips for Better Code, UX, and Performance

1. Use const constructors where possible

This reduces unnecessary widget rebuilds and enhances performance, especially in widget-heavy trees.

2. Prefer StateNotifier/Riverpod or Cubit

State management is more scalable and predictable compared to relying on setState in larger apps.

3. Avoid using context after async gaps

Calling context after awaiting an async function can lead to crashes if the widget is unmounted.

4. Use MediaQuery/LayoutBuilder for responsiveness

These tools help your UI adapt properly across screen sizes and orientations.

5. Create a reusable design system

Centralizing colors, typography, and spacing keeps your UI consistent and easier to update.

6. Avoid overusing FutureBuilder/StreamBuilder

Too much logic in widgets reduces readability. Shift data fetching to view models or providers.

7. Write widget and unit tests

Helps catch regressions and maintain stability in critical UI and business logic.

8. Handle nulls explicitly

Avoid null-related crashes by using non-nullable types, default values, and required parameters.

9. Use flutter_gen or build_runner for assets

Avoid typo-related bugs in asset names by generating constants automatically.

10. Use keys smartly in lists and forms

Keys help Flutter track widgets efficiently and prevent visual glitches.

11. Split logic-heavy widgets

60+ Pro Flutter Tips for Better Code, UX, and Performance

Widgets with business logic should offload it to view models or service layers for clarity.

12. Use `flutter_lints` or `very_good_analysis`

Automatically enforce coding standards and detect common issues early.

13. Cache images and assets

Improve performance by preloading or caching frequently used resources.

14. Use `showModalBottomSheet` with `isScrollControlled`

Enhance UX by allowing full-screen bottom sheets when needed.

15. Use Animated widgets

Declarative animations like `AnimatedOpacity` or `AnimatedContainer` are cleaner and less error-prone.

16. Use extension methods

Extensions make your code cleaner and encapsulate common widget helpers.

17. Create `AppTheme` and `AppTextStyles`

Organizing themes into custom classes helps maintain scalable design systems.

18. Use `Sliver` widgets for scroll behavior

Slivers allow advanced scroll effects like collapsible headers and lazy loading.

19. Debounce user input

Avoid calling expensive operations on every keystroke, especially in search fields.

20. Use `LayoutBuilder` for adaptive layouts

Build responsive UIs by dynamically adjusting widgets based on constraints.

21. Use `intl` for localization and formatting

Support multiple languages and regions using the `intl` package.

60+ Pro Flutter Tips for Better Code, UX, and Performance

22. Profile with Flutter DevTools

Identify performance issues like rebuilds, memory leaks, and jank.

23. Avoid deep nesting

Deeply nested widgets hurt readability and are harder to debug. Extract them instead.

24. Use ThemeExtension for custom theme values

Add custom colors, spacings, or shadows in a scalable way.

25. Use builder in showDialog/showModal

Prevents issues related to context and stale widget trees.

26. Always check if mounted before setState

Protects against calling setState after the widget has been disposed.

27. Use AutovalidateMode.onUserInteraction

Provides better user experience by validating only when necessary.

28. Use const and child to optimize rebuilds

Pass const widgets as children to avoid unnecessary updates.

29. Preserve tab state with IndexedStack

Keep tab contents alive when switching between bottom navigation items.

30. Extract Theme.of/MediaQuery calls early

Improves code readability and avoids redundant calls.

31. Avoid async directly in initState

Dart disallows async initState, so handle setup in separate async methods.

32. Know when to use GestureDetector vs InkWell

60+ Pro Flutter Tips for Better Code, UX, and Performance

InkWell adds material ripple, GestureDetector is more flexible but less visual.

33. Use flutter_hooks for less boilerplate

Hooks simplify lifecycle management in widgets.

34. Group imports for readability

Organize system, package, and project imports separately.

35. Use clipBehavior for smoother corners

Ensures proper rendering of rounded corners and shadows.

36. Use ListView.separated

Easier and cleaner than manually inserting spacers between list items.

37. Wrap heavy widgets with RepaintBoundary

Isolates redraws for performance improvements.

38. Use AnimatedSwitcher for smooth transitions

Fade between widgets on state change for better UI.

39. Group environment configs

Use flavors to manage dev, staging, and prod builds safely.

40. Use enum + switch for logic

Avoids string literals and makes logic more readable and maintainable.

41. Document public widgets

Improve maintainability by explaining what your components do.

42. Disable Android scroll glow

Provides a more refined look in custom scroll views.

60+ Pro Flutter Tips for Better Code, UX, and Performance

43. Automate with GitHub Actions or Codemagic

Use CI/CD pipelines to automate tests and deployments.

44. Use freezed for data classes

Avoid boilerplate with immutable models and sealed unions.

45. Use keys for preserving widget state

Ensures proper state retention, especially in dynamic lists.

46. Avoid placing large objects in build method

Move object creation to initState or external providers.

47. Modularize your app structure

Split your app into features/modules for scalability.

48. Use const constructors in widgets

Improves rendering efficiency and build optimization.

49. Use proper logging instead of print

Use packages like logger for structured output.

50. Use assert() for developer checks

Catch incorrect assumptions early in development.

51. Avoid putting business logic in UI

UI should be dumb - delegate logic to services or view models.

52. Use Navigator 2.0 for advanced routing

Offers more flexibility for web, auth flows, and deep linking.

53. Keep pubspec.yaml organized

60+ Pro Flutter Tips for Better Code, UX, and Performance

Group and comment your dependencies for better clarity.

54. Leverage platform channels wisely

Bridge Flutter with native code only when absolutely needed.

55. Monitor app size regularly

Keep track of size to optimize load time and memory use.

56. Use packages with good maintenance

Check update frequency, issue count, and pub score before using.

57. Use debounce in form fields

Avoid unnecessary rebuilds or validations on every key press.

58. Add semantic labels for accessibility

Improves screen reader support and inclusivity.

59. Use native splash screens

Ensure smooth and fast app start-up experience.

60. Use lifecycle-aware widgets

Track app lifecycle for things like analytics and sessions.

61. Use SharedPreferences or secure storage wisely

Persist only essential or non-sensitive data locally.